

Wydział: W4N
Rok: 2025/2026
Semestr: 5

Data oddania pracy:
3 stycznia 2026

SYSTEMY OPERACYJNE

Część I: Dwukierunkowy dwuwątkowy algorytm Dijkstry

Część II: Uczujący filozofowie współbieżność

Autor:

Kacper Piekut 280879

Prowadzący:

dr Marek Bazan



Politechnika Wrocławska

1. Część I: Dwukierunkowy dwuwątkowy algorytm Dijkstry

1.1. Wstęp

Algorytm Dijkstry jest jednym z najważniejszych i najczęściej stosowanych algorytmów grafowych służących do wyznaczania najkrótszych ścieżek w grafach o nieujemnych wagach krawędzi. Znajduje szerokie zastosowanie w informatyce, m.in. w systemach nawigacyjnych, sieciach komputerowych czy optymalizacji tras. Klasyczna wersja algorytmu działa w sposób jednowątkowy, co może prowadzić do ograniczeń wydajnościowych przy przetwarzaniu dużych grafów. W odpowiedzi na te wyzwania powstały różne podejścia do równoległego przetwarzania, w tym dwukierunkowy algorytm Dijkstry, który wykorzystuje dwa wątki do jednoczesnego poszukiwania najkrótszej ścieżki od źródła do celu oraz od celu do źródła. Takie podejście może znacząco przyspieszyć proces znajdowania najkrótszej ścieżki, zwłaszcza w dużych i złożonych grafach.

1.2. Cele projektu

Celem projektu jest zaimplementowanie i przetestowanie algorytmu Dijkstry w wersji jednowątkowej oraz dwuwątkowej w języku C++. Projekt zakłada:

- stworzenie generatora losowych grafów skierowanych o zadanych parametrach,
- implementację klasycznego algorytmu Dijkstry oraz jego wersji dwuwątkowej,
- pomiar i porównanie czasów działania obu wersji algorytmu dla różnych rozmiarów grafów i gęstości krawędzi,
- analizę poprawności wyznaczanych ścieżek i wyników,
- ocenę korzyści z zastosowania wielowątkowości w praktyce.

1.3. Plan eksperymentu

Testowany algorytm

Testowany jest klasyczny algorytm Dijkstry w wersji jednowątkowej oraz jego wariant dwuwątkowy. Obie wersje korzystają z kolejki priorytetowej i operują na grafie skierowanym o nieujemnych wagach krawędzi. Algorytmy wyznaczają najkrótszą ścieżkę pomiędzy wybranymi wierzchołkami grafu.

Dane wejściowe i generowanie grafów

Dane wejściowe stanowią losowo generowane grafy skierowane o zadanej liczbie wierzchołków i gęstości krawędzi w postaci macierzowej reprezentacji. Grafy są generowane automatycznie przez program. Wagi krawędzi są losowane z ustalonego zakresu liczb całkowitych.

Środowisko testowe

Eksperyment jest wykonywany w środowisku konsolowym na komputerze z systemem Windows. Kod został napisany w języku C++ i kompilowany przy użyciu kompilatora MinGW (GCC) w wersji 13.2.0. Wyniki mogą być zależne od wydajności sprzętu, dlatego wnioski dotyczące efektywności algorytmów są formułowane w sposób ogólny.

Specyfikacja sprzętowa

Laptop: ASUS TUF Gaming F15.

Procesor: Intel Core i5-11400H, 6 rdzeni, 12 wątków, taktowanie bazowe 2.7 GHz (do 4.5 GHz w trybie turbo).

Pamięć RAM: 16 GB DDR4.

Dysk: SSD NVMe 512 GB (Samsung MZVLQ512HBLU-00B00).

Karta graficzna: NVIDIA GeForce RTX 3050 Laptop GPU oraz Intel UHD Graphics.

System operacyjny: Microsoft Windows 11 Home 64-bit, wersja 10.0.26100.

1.4. Implementacja

Program testuje algorytm Dijkstry w wersji jednowątkowej i dwuwątkowej, korzystając z biblioteki STL oraz mechanizmów wielowątkowości C++.

Zmienne globalne:

- **INF** - stała oznaczająca nieskończoność (brak ścieżki),
- **path_single, path_double** - wektory przechowujące wyznaczone ścieżki dla wersji jednowątkowej i dwuwątkowej.

Funkcja generateGraph - generuje skierowany graf o liczbie wierzchołków n i gęstości d w macierzowej reprezentacji (`vector<vector<int>`), gdzie INF oznacza brak krawędzi, a na przekątnej znajdują się zera. Funkcja wspiera tryb siatki (grid):

- w trybie grid budowana jest siatka 2D z krawędziami tylko w prawo i w dół (skierowane), wagi losowane z przedziału $[1, n]$;
- następnie dobija się wymaganą gęstość losowymi krawędziami o wagach z przedziału $[1, 2n]$ (większy zakres dla krawędzi dodatkowych);
- w trybie losowym (bez gridu) generowane są krawędzie z wagami $[1, n]$.

Funkcja pomocnicza reconstruct_path - rekonstruuje ścieżkę z tablic rodziców wyznaczonych przez przeszukiwania od początku i od końca.

Funkcja dijkstraSingle - jednowątkowa wersja z dwukierunkowym przeszukiwaniem. Budowana jest odwrócona macierz, utrzymywane są dwie kolejki priorytetowe (front od początku i od końca), a warunek szybkiego zakończenia porównuje najmniejsze klucze w kolejkach z aktualnym best (najlepszym znalezionym dystansem). Po wyznaczeniu punktu spotkania meet ścieżka jest rekonstruowana.

Funkcja dijkstraDouble - wersja dwuwątkowa. Dwa wątki niezależnie przeszukują graf (macierz i jej odwrócenie), współdzieląc best, meet i flagę stop. Synchronizacja tych zmiennych odbywa się za pomocą mutex; wątek może zakończyć pracę wcześniej, gdy dalsze ulepszanie best nie jest możliwe.

Funkcja measureTime - mierzy czas wykonania przekazanej funkcji i zwraca wynik w milisekundach (`high_resolution_clock`).

Funkcja main - inicjalizacja, pętle testowe dla zestawów n i d , włączenie trybu siatki (grid), ustawienie `start=0`, `end=n-1` w trybie grid, uruchomienie obu wersji algorytmu, weryfikacja poprawności (porównanie dystansów), agregacja i wypis wyników (w tym proporcji czasu jednowątkowego do dwuwątkowego).

Struktura danych:

- Graf: `vector<vector<int>>` - macierz sąsiedztwa z wagami; INF oznacza brak krawędzi. Dla przeszukiwania od końca budowana jest odwrócona macierz.
- Kolejki priorytetowe: `priority_queue<pair<int,int>, vector<pair<int,int>, greater<pair<int,int>>>` - wybór wierzchołka o najmniejszej odległości w obu kierunkach.
- Tablice odległości i rodziców: `vector<int>` - przechowują aktualne odległości od początku/końca oraz poprzedników na ścieżce.

Wielowątkowość:

- Wersja dwuwątkowa używa `std::thread` do równoległego uruchomienia dwóch przeszukiwań; kluczowe zmienne współdzielone są chronione przez mutex.
- Po zakończeniu pracy wątków wykonywany jest `join()`, a następnie następuje rekonstrukcja ścieżki.

Obsługa pomiarów i prezentacja wyników:

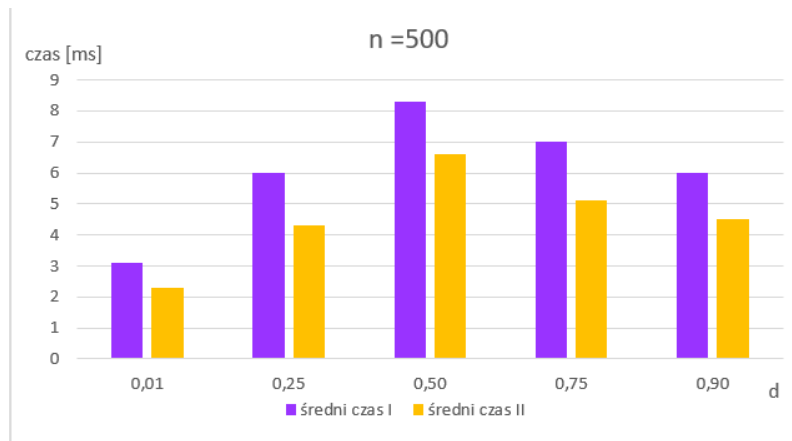
- Dla każdej konfiguracji (rozmiar grafu, gęstość) wykonywanych jest 10 prób, a wyniki są uśredniane.
- Program wypisuje na konsolę średnie czasy działania obu wersji algorytmu dla każdej konfiguracji.
- Opcjonalnie możliwe jest wypisanie wygenerowanego grafu oraz wyznaczonych ścieżek.

1.5. Badania

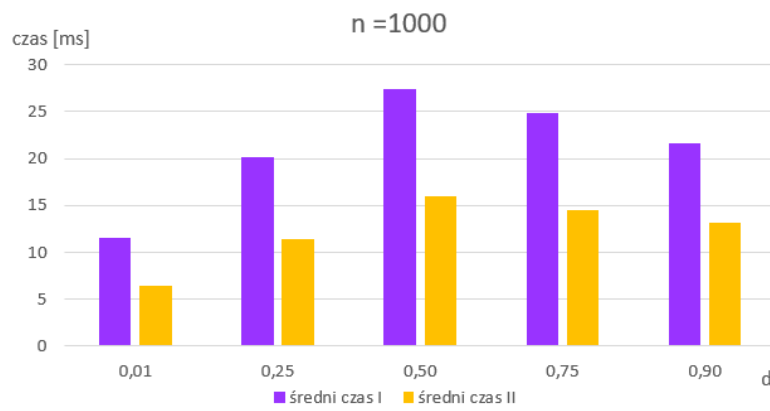
W celu porównania wydajności klasycznego (jednowątkowego) oraz dwuwątkowego algorytmu Dijkstry przeprowadzono serię testów na losowo generowanych grafach skierowanych o różnych rozmiarach (n) i gęstościach (d). Dla każdej konfiguracji przeprowadzono po 10 testów, których został wyciągnięty średni czas wykonania się algorytmu obu wersji. Wyniki przedstawiono w poniższej tabeli.

Liczba wierzchołków n	Gęstość d	Średni czas I [ms]	Średni czas II [ms]	Różnica
500	0.01	3.1	2.3	1.34
500	0.25	6.0	4.3	1.39
500	0.50	8.3	6.6	1.25
500	0.75	7.0	5.1	1.37
500	0.90	6.0	4.5	1.33
1000	0.01	11.5	6.5	1.76
1000	0.25	20.2	11.4	1.77
1000	0.50	27.4	16.0	1.71
1000	0.75	24.9	14.5	1.71
1000	0.90	21.6	13.2	1.63
5000	0.01	270.2	164.8	1.63
5000	0.25	488.6	296.1	1.65
5000	0.50	700.8	425.2	1.64
5000	0.75	615.3	388.1	1.58
5000	0.90	543.3	376.4	1.44
10000	0.01	1169.8	862.2	1.35
10000	0.25	2274.5	1677.8	1.35
10000	0.50	3210.2	2282.2	1.4
10000	0.75	2913.7	2152.0	1.35
10000	0.90	2706.1	2103.4	1.28
20000	0.01	4903.8	3633.9	1.34
20000	0.25	10352.1	7770.0	1.32
20000	0.50	13335.8	10318.9	1.29
20000	0.75	14053.5	10724.6	1.31
20000	0.90	13080.4	10560	1.23

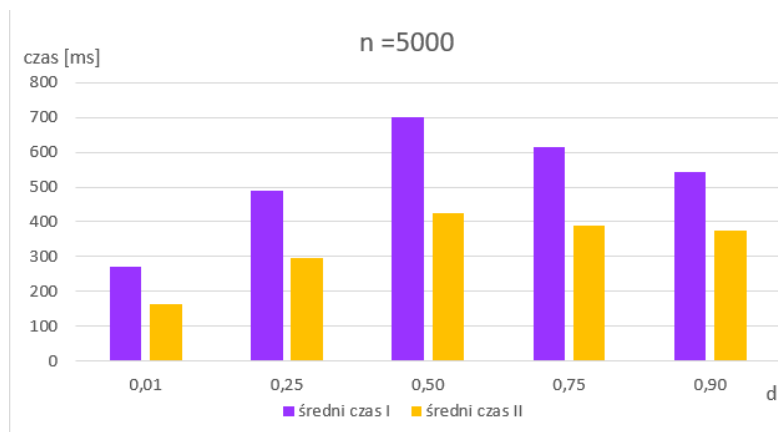
Tabela 1: Porównanie średnich czasów wykonania algorytmu Dijkstry w wersji jednowątkowej (I) i dwuwątkowej (II) dla różnych rozmiarów i gęstości grafu oraz stosunek I/II (Różnica).



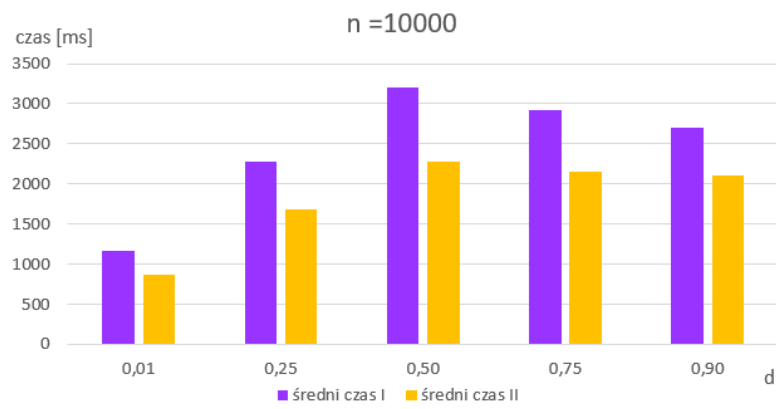
Rysunek 1: Porównanie czasów dla $n = 500$



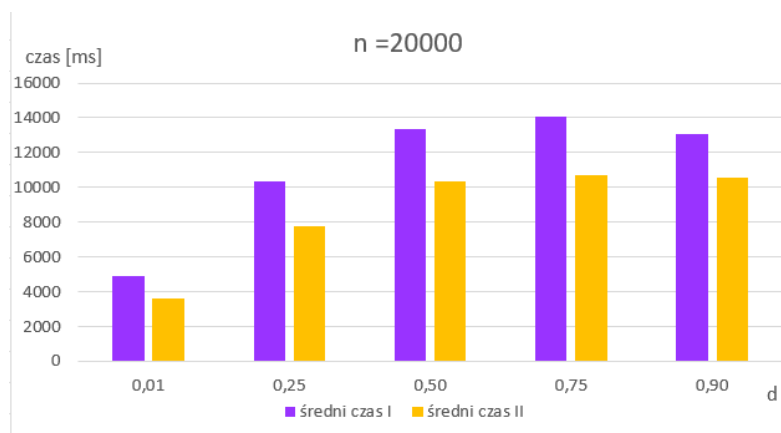
Rysunek 2: Porównanie czasów dla $n = 1000$



Rysunek 3: Porównanie czasów dla $n = 5000$



Rysunek 4: Porównanie czasów dla $n = 10000$



Rysunek 5: Porównanie czasów dla $n = 20000$

1.6. Wnioski

Wyniki aktualnych pomiarów pokazują, że dwukierunkowa wersja algorytmu jest konsekwentnie szybsza od jednowątkowej we wszystkich badanych konfiguracjach. Zarejestrowane proporcje czasu I/II mieszczą się najczęściej w przedziale 1.3-1.7, a maksymalny odnotowany zysk wyniósł **1.77** (dla $n = 1000$, $d = 0,25$).

- **Wpływ gęstości:** największe przyspieszenia uzyskiwano dla umiarkowanej gęstości (np. $d = 0,25$). Dla bardzo rzadkich ($d = 0,01$) oraz bardzo gęstych ($d = 0,9$) grafów zysk był mniejszy (zwykle 1.2-1.35). Wynika to z krótszych tras i szybszego domknięcia best, co ogranicza zakres równoległego przeszukiwania.
- **Wpływ rozmiaru:** wraz ze wzrostem n czasy rosną, ale stosunek I/II pozostaje stabilny i najczęściej mieści się w zakresie 1.3-1.6. Przy największym rozmiarze ($n = 20000$) dla $d = 0,25$ zanotowano zysk na poziomie 1.33, a dla $d = 0,75$ 1.31.
- **Jednowątkowy vs. narzuty:** przy małych n różnice są ograniczone przez narzuty uruchomienia wątków i synchronizacji, jednak nadal obserwuje się przewagę wersji dwuwątkowej.
- **Struktura grafu:** tryb grid (krawędzie prawo/dół, wagi $[1, n]$) wymusza trasy z większą liczbą wyrzchołków na trasie, co sprzyja dwukierunkowości. Dodatkowe krawędzie o większym zakresie wag rzadziej tworzą opłacalne skróty.

1.7. Podsumowanie

Zaimplementowano dwie wersje algorytmu Dijkstry (jednowątkową i dwuwątkową - dwukierunkowe) działające na macierzowej reprezentacji grafu. Obie wersje zwracają zgodne długości najkrótszych ścieżek, a pomiary potwierdzają systematyczny zysk czasowy wariantu dwuwątkowego - najczęściej 1.3-1.7, lokalnie do **1.77**. Skuteczność rośnie dla umiarkowanych gęstości i większych rozmiarów, maleje dla skrajnych gęstości.

Ograniczenia w osiąganiu pełnego dwukrotnego przyśpieszenia wynikają m.in. z narzutów synchronizacji, kosztów operacji na `priority_queue` oraz skanowania wierszy macierzy (koszt per wierzchołek niezależny od stopnia).

1.8. Literatura

- https://pl.wikipedia.org/wiki/Algorytm_Dijkstry
- https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
- Mirosław Zelent kurs C++ <https://miroslawzelent.pl/kurs-c++/>

1.9. Kod

```
1  #include <iostream>
2  #include <vector>
3  #include <cstdlib>
4  #include <ctime>
5  #include <queue>
6  #include <thread>
7  #include <mutex>
8  #include <chrono>
9  #include <functional>
10 #include <algorithm>
11 #include <cmath>
12
13 std::vector<int> path_single;
14 std::vector<int> path_double;
15
16 using std::cout;
17 using std::vector;
18 using std::pair;
19 using std::make_pair;
20 using std::priority_queue;
21 using std::greater;
22 using std::thread;
23 using std::mutex;
24 using std::lock_guard;
25 using std::chrono::high_resolution_clock;
26
27 int INF = 2000000000; // reprezentacja
    ↪ nieskończoności
28
29 vector<vector<int>> generateGraph(int n, double d, bool show_graph,
    ↪ bool grid_mode) {
30     vector<vector<int>> adj_matrix(n, vector<int>(n, INF));
31     for (int i = 0; i < n; ++i) adj_matrix[i][i] = 0;
32     int max_edges = n * (n - 1); // maksymalna
    ↪ liczba krawędzi -> graf skierowany, bez pętli
33     int m = (int)(d * max_edges + 0.5); // m - liczba
    ↪ krawędzi
34     vector<bool> edge_exists(n * n, false);
35     int added = 0; // licznik dodanych krawędzi
36     // generator losowy siatki
37     if (grid_mode) {
38         // wymiary siatki możliwie zbliżone do kwadratu
```

```

39     int rows = (int)std::floor(std::sqrt((double)n));
40     if (rows < 1) rows = 1;
41     int cols = (int)std::ceil((double)n / rows);
42     // krawędzie siatki: tylko w prawo i w dół (skierowane),
    ↪ wagi 1..n
43     for (int id = 0; id < n; ++id) {
44         int r = id / cols;
45         int c = id % cols;
46         // w prawo
47         if (c + 1 < cols) {
48             int v = id + 1;
49             if (v < n) {
50                 int idx = id * n + v;
51                 if (!edge_exists[idx]) {
52                     edge_exists[idx] = true;
53                     int waga = 1 + (rand() % n);
54                     adj_matrix[id][v] = waga;
55                     ++added;
56                 }
57             }
58         }
59         // w dół
60         if (r + 1 < rows) {
61             int v = id + cols;
62             if (v < n) {
63                 int idx = id * n + v;
64                 if (!edge_exists[idx]) {
65                     edge_exists[idx] = true;
66                     int waga = 1 + (rand() % n);
67                     adj_matrix[id][v] = waga;
68                     ++added;
69                 }
70             }
71         }
72     }
73     // dobijanie gęstości losowymi krawędziami o większych
    ↪ wagach
74     while (added < m) {
75         int u = rand() % n;
76         int v = rand() % n;
77         if (u == v) continue;
78         int idx = u * n + v;
79         if (edge_exists[idx]) continue;
80         edge_exists[idx] = true;
81         int waga2 = (n * n) + (rand() % (n * n * n));
82         adj_matrix[u][v] = waga2;
83         ++added;
84     }
85 } else {

```

```

86         // poprzedni generator losowy
87         while (added < m) {
88             int u = rand() % n;                // losowy wierzchołek
89             ↪ początkowy
90             int v = rand() % n;                // losowy wierzchołek
91             ↪ końcowy
92             if (u == v) continue;
93             int idx = u * n + v;
94             if (edge_exists[idx]) continue;
95             edge_exists[idx] = true;
96             int waga = 1 + (rand() % n);      // losowa waga krawędzi
97             ↪ z zakresu [1, n]
98             adj_matrix[u][v] = waga;
99             ++added;
100         }
101     }
102     if (show_graph) {
103         cout << "Wygenerowany graf (n=" << n << ", m=" << m <<
104         ↪ "):\n";
105         for (int u = 0; u < n; ++u) {
106             cout << u << ":";
107             for (int v = 0; v < n; ++v) {
108                 if (u != v && adj_matrix[u][v] < INF) {
109                     cout << " (" << v << ", " << adj_matrix[u][v] <<
110                     ↪ ")";
111                 }
112             }
113             cout << "\n";
114         }
115     }
116     return adj_matrix;
117 }
118
119 // rekonstrukcja ścieżki
120 static void reconstruct_path(vector<int>& parent_start, vector<int>&
121 ↪ parent_end, int meet, vector<int>& out_path) {
122     out_path.clear();
123     if (meet == -1) return;
124     vector<int> left, right;
125     int v = meet;
126     while (v != -1) {
127         left.push_back(v); v = parent_start[v];
128     }
129     reverse(left.begin(), left.end());
130     v = parent_end[meet];
131     while (v != -1) {
132         right.push_back(v); v = parent_end[v];
133     }
134     out_path = left;

```

```

129     out_path.insert(out_path.end(), right.begin(), right.end());
130 }
131
132 int dijkstraSingle(vector<vector<int>> &adj_matrix, int start, int
↪ end) {
133     int n = (int)adj_matrix.size();
134     // odwrócony graf jako macierz
135     vector<vector<int>> rev_matrix(n, vector<int>(n, INF));
136     for (int u = 0; u < n; ++u) {
137         for (int v = 0; v < n; ++v) {
138             if (adj_matrix[u][v] < INF) rev_matrix[v][u] =
↪ adj_matrix[u][v];
139         }
140     }
141     vector<int> dist_start(n, INF), dist_end(n, INF);
142     vector<int> parent_start(n, -1), parent_end(n, -1);
143     dist_start[start] = 0;
144     dist_end[end] = 0;
145     priority_queue<pair<int, int>, vector<pair<int, int>>,
↪ greater<pair<int, int>>> pq_start, pq_end;
146     pq_start.push(make_pair(0, start));
147     pq_end.push(make_pair(0, end));
148     int best = INF, meet = -1;
149     while (true) {
150         if (pq_start.empty() && pq_end.empty()) break;
151         // krok od początku
152         if (!pq_start.empty()) {
153             int d = pq_start.top().first;
154             int u = pq_start.top().second;
155             pq_start.pop();
156             if (d <= dist_start[u]) {
157                 // aktualizacja best/meet na przecięciu
158                 if (dist_end[u] < INF) {
159                     int candidate = dist_start[u] + dist_end[u];
160                     if (candidate < best) { best = candidate; meet =
↪ u; }
161                 }
162                 for (int v = 0; v < n; ++v) {
163                     int w = adj_matrix[u][v];
164                     if (w < INF && dist_start[v] > dist_start[u] +
↪ w) {
165                         dist_start[v] = dist_start[u] + w;
166                         parent_start[v] = u;
167                         pq_start.push(make_pair(dist_start[v], v));
168                         if (dist_end[v] < INF) {
169                             int candidate2 = dist_start[v] +
↪ dist_end[v];
170                             if (candidate2 < best) { best =
↪ candidate2; meet = v; }

```

```

171         }
172     }
173 }
174 }
175 }
176 // krok od końca
177 if (!pq_end.empty()) {
178     int d = pq_end.top().first;
179     int u = pq_end.top().second;
180     pq_end.pop();
181     if (d <= dist_end[u]) {
182         if (dist_start[u] < INF) {
183             int candidate = dist_end[u] + dist_start[u];
184             if (candidate < best) { best = candidate; meet =
185                 ↪ u; }
186         }
187         for (int v = 0; v < n; ++v) {
188             int w = rev_matrix[u][v];
189             if (w < INF && dist_end[v] > dist_end[u] + w) {
190                 dist_end[v] = dist_end[u] + w;
191                 parent_end[v] = u;
192                 pq_end.push(make_pair(dist_end[v], v));
193                 if (dist_start[v] < INF) {
194                     int candidate2 = dist_end[v] +
195                         ↪ dist_start[v];
196                     if (candidate2 < best) { best =
197                         ↪ candidate2; meet = v; }
198                 }
199             }
200         }
201     }
202     // warunek szybkiego zakończenia
203     bool canImproveStart = !pq_start.empty() &&
204         ↪ pq_start.top().first < best;
205     bool canImproveEnd = !pq_end.empty() &&
206         ↪ pq_end.top().first < best;
207     if (!canImproveStart && !canImproveEnd) break;
208 }
209 reconstruct_path(parent_start, parent_end, meet, path_single);
210 return best < INF ? best : INF;
211 }
212
213 int dijkstraDouble(vector<vector<int>> &adj_matrix, int start, int
214     ↪ end) {
215     int n = (int)adj_matrix.size();
216     // odwrócony graf jako macierz
217     vector<vector<int>> rev_matrix(n, vector<int>(n, INF));
218     for (int u = 0; u < n; ++u) {

```

```

214         for (int v = 0; v < n; ++v) {
215             if (adj_matrix[u][v] < INF) rev_matrix[v][u] =
                ↪ adj_matrix[u][v];
216         }
217     }
218     vector<int> dist_start(n, INF), dist_end(n, INF);
219     vector<int> parent_start(n, -1), parent_end(n, -1);
220     dist_start[start] = 0;
221     dist_end[end] = 0;
222     priority_queue<pair<int, int>, vector<pair<int, int>>,
        ↪ greater<pair<int, int>>> pq_start, pq_end;
223     pq_start.push(make_pair(0, start));
224     pq_end.push(make_pair(0, end));
225     // wspólne zmienne
226     int best = INF;
227     int meet = -1;
228     bool stop = false;
229     mutex mtx_shared;
230     auto search = [&](vector<int> &dist, vector<int> &parent,
        ↪ priority_queue<pair<int, int>, vector<pair<int, int>>,
        ↪ greater<pair<int, int>>> &pq, vector<vector<int>> &G, const
        ↪ vector<int> &otherDist) {
231         int nLocal = (int)G.size();
232         while (true) {
233             {
234                 lock_guard<mutex> lk(mtx_shared);
235                 if (stop) break;
236             }
237             if (pq.empty()) break;
238             int d = pq.top().first;
239             int u = pq.top().second;
240             pq.pop();
241             if (d > dist[u]) continue;
242             {
243                 lock_guard<mutex> lk(mtx_shared);
244                 if (otherDist[u] < INF) {
245                     int candidate = dist[u] + otherDist[u];
246                     if (candidate < best) {
247                         best = candidate;
248                         meet = u;
249                     }
250                 }
251             }
252             for (int v = 0; v < nLocal; ++v) {
253                 int w = G[u][v];
254                 if (w < INF && dist[v] > dist[u] + w) {
255                     dist[v] = dist[u] + w;
256                     parent[v] = u;
257                     pq.push(make_pair(dist[v], v));

```



```

258         {
259             lock_guard<mutex> lk(mtx_shared);
260             if (otherDist[v] < INF) {
261                 int candidate = dist[v] + otherDist[v];
262                 if (candidate < best) {
263                     best = candidate;
264                     meet = v;
265                 }
266             }
267         }
268     }
269 }
270 {
271     lock_guard<mutex> lk(mtx_shared);
272     if (!pq.empty() && pq.top().first >= best) {
273         stop = true;
274     }
275 }
276 }
277 };
278 // uruchomienie wątków
279 thread t1(search, ref(dist_start), ref(parent_start),
    ↪ ref(pq_start), ref(adj_matrix), ref(dist_end));
280 thread t2(search, ref(dist_end), ref(parent_end), ref(pq_end),
    ↪ ref(rev_matrix), ref(dist_start));
281 t1.join();
282 t2.join();
283 reconstruct_path(parent_start, parent_end, meet, path_double);
284 return best < INF ? best : INF;
285 }
286
287 double measureTime(const std::function<void()>& func) {
288     auto t1 = high_resolution_clock::now();
289     func();
290     auto t2 = high_resolution_clock::now();
291     return std::chrono::duration_cast<std::chrono::milliseconds>(t2
    ↪ - t1).count();
292 }
293
294 int main() {
295     srand((unsigned)time(nullptr)); // ziarno dla generatora liczb
    ↪ losowych
296     bool show_graph = false; // drukowanie grafu
297     bool show_path = false; // wypisanie ścieżki i
    ↪ odległości
298     const int Ns[] = {500, 1000, 5000, 10000, 20000};
299     const double Ds[] = {0.01, 0.25, 0.5, 0.75, 0.9};
300     const int numN = sizeof(Ns) / sizeof(Ns[0]);
301     const int numD = sizeof(Ds) / sizeof(Ds[0]);

```

```

302     const int trials = 10;
303     for (int iN = 0; iN < numN; ++iN) {
304         int n = Ns[iN];
305         for (int iD = 0; iD < numD; ++iD) {
306             double d = Ds[iD];
307             double sum_single = 0.0;
308             double sum_double = 0.0;
309             int good_trials = 0;
310             int attempts = 0;
311             while (good_trials < trials) {
312                 ++attempts;
313                 bool grid_mode = true; // tryb siatki
314                 vector<vector<int>> adj_matrix = generateGraph(n, d,
↪ show_graph, grid_mode);
315                 int start, end;
316                 if (grid_mode) {
317                     start = 0;
318                     end = n - 1;
319                 } else {
320                     start = rand() % n;
321                     end = rand() % n;
322                     while (end == start) end = rand() % n;
323                 }
324                 int dist_single;
325                 double time_single = measureTime([&]() {
326                     dist_single = dijkstraSingle(adj_matrix, start,
↪ end);
327                 });
328                 int dist_double;
329                 double time_double = measureTime([&]() {
330                     dist_double = dijkstraDouble(adj_matrix, start,
↪ end);
331                 });
332                 if (dist_single == dist_double) {
333                     sum_single += time_single;
334                     sum_double += time_double;
335                     ++good_trials;
336                     if (show_path) {
337                         cout << "[Proba " << (good_trials) << "] n="
↪ << n << ", d=" << d << ", start=" <<
↪ start << ", end=" << end << "\n";
338                         cout << "Jednowatkowy: odleglosc = ";
339                         if (dist_single == INF) {
340                             cout << "brak sciezki";
341                         } else {
342                             cout << dist_single << ", sciezka: ";
343                             for (size_t i = 0; i <
↪ path_single.size(); ++i) {
344                                 cout << path_single[i];

```

```

345         if (i + 1 < path_single.size()) cout
           ↪ << " -> ";
346     }
347 }
348 cout << "\n";
349 cout << "Dwuwatkowy: odleglosc= ";
350 if (dist_double == INF) {
351     cout << "brak sciezki";
352 } else {
353     cout << dist_double << ", sciezka: ";
354     for (size_t i = 0; i <
           ↪ path_double.size(); ++i) {
355         cout << path_double[i];
356         if (i + 1 < path_double.size()) cout
           ↪ << " -> ";
357     }
358 }
359 cout << "\n";
360 cout << "Czas jednowatkowy = " <<
           ↪ time_single << " ms\n";
361 cout << "Czas dwuwatkowy = " << time_double
           ↪ << " ms\n";
362 }
363 } else {
364     if (show_path) {
365         cout << "[Proba odrzucona]\n";
366     }
367 }
368 }
369 double avg_single = sum_single / trials;
370 double avg_double = sum_double / trials;
371 double proportion = (double)((int)(avg_single/avg_double
           ↪ * 100.0)) / 100.0;
372 cout << "n=" << n << ", d=" << d
373     << ": sredni czas jednowatkowy = " << avg_single <<
           ↪ " ms, "
374     << "dwuwatkowy = " << avg_double << " ms, roznica:
           ↪ " << proportion << "\n";
375     }
376 }
377 return 0;
378 }

```

2. Część II: Uczujący filozofowie współbieżność

2.1. Wstęp

Problem uczujących filozofów jest jednym z klasycznych zagadnień synchronizacji w programowaniu współbieżnym. Został sformułowany przez Edsgera Dijkstrę jako ilustracja problemów związanych z dostępem do współdzielonych zasobów przez wiele procesów. Uczujący filozofowie są metaforą dla procesów konkurujących o ograniczone zasoby, takie jak np. drukarki, pliki czy inne urządzenia peryferyjne. Niewłaściwa synchronizacja może prowadzić do sytuacji, w której żaden z procesów nie może kontynuować pracy (zakleszczenie) lub niektóre procesy są stale pomijane (zagłodzenie). Analiza i implementacja tego problemu pozwala lepiej zrozumieć mechanizmy synchronizacji, takie jak semaforey czy muteksy czy, oraz uczy projektowania algorytmów odpornych na typowe błędy współbieżności. Wersja zadania zakłada, że N filozofów siedzi przy okrągłym stole, a pomiędzy każdym z nich leży jeden widelec. Każdy filozof naprzemiennie myśli i je, a do jedzenia potrzebuje dwóch widelców - lewego i prawego.

2.2. Cele projektu

Celem projektu jest zaimplementowanie wielowątkowej symulacji problemu uczujących filozofów w języku C++. Program ma umożliwiać obserwację zachowania filozofów oraz poprawnie synchronizować dostęp do współdzielonych zasobów (widelców), tak aby:

- zapobiec zakleszczeniu,
- zminimalizować ryzyko zagłodzenia,
- umożliwić równoległe działanie wielu wątków,
- zilustrować praktyczne aspekty synchronizacji w programowaniu współbieżnym.

2.3. Plan eksperymentu

Testowany kod i scenariusz

Testowany jest program symulujący problem uczujących filozofów, w którym każdy filozof jest reprezentowany przez osobny wątek. Synchronizacja dostępu do widelców realizowana jest za pomocą mutexów oraz własnej implementacji semafora licznikowego. Program umożliwia wykrywanie zagłodzenia.

Dane wejściowe i parametry

Dane wejściowe stanowią liczba filozofów (domyślnie 100), czas trwania symulacji oraz parametry dotyczące czasu myślenia i jedzenia (losowane z ustalonych zakresów). Program umożliwia modyfikację tych parametrów w celu obserwacji różnych scenariuszy synchronizacji i ryzyka zagłodzenia.

Środowisko testowe

Eksperyment jest wykonywany w środowisku konsolowym na komputerze z systemem Windows. Kod został napisany w języku C++ i kompilowany przy użyciu kompilatora MinGW (GCC) w wersji 13.2.0. Wyniki mogą być zależne od wydajności sprzętu, dlatego wnioski dotyczące efektywności algorytmów są formułowane w sposób ogólny.

Specyfikacja sprzętowa

Laptop: ASUS TUF Gaming F15.

Procesor: Intel Core i5-11400H, 6 rdzeni, 12 wątków, taktowanie bazowe 2.7 GHz (do 4.5 GHz w trybie turbo).

Pamięć RAM: 16 GB DDR4.

Dysk: SSD NVMe 512 GB (Samsung MZVLQ512HBLU-00B00).

Karta graficzna: NVIDIA GeForce RTX 3050 Laptop GPU oraz Intel UHD Graphics.

System operacyjny: Microsoft Windows 11 Home 64-bit, wersja 10.0.26100.

2.4. Implementacja

Implementacja problemu uczujących filozofów została zrealizowana w języku C++ z wykorzystaniem wątków oraz mechanizmów synchronizacji dostępnych w standardowej bibliotece. Każdy filozof jest reprezentowany przez osobny wątek, który cyklicznie przechodzi przez stany: myślenie, głód oraz jedzenie.

Główne zmienne i struktury:

- `int threads` - liczba filozofów oraz widelców (domyślnie 100).
- `vector<thread> thread_vec` - wektor przechowujący obiekty wątków, po jednym dla każdego filozofa.
- `vector<mutex> sticks` - wektor mutexów reprezentujących widelce leżące pomiędzy filozofami.
- `atomic_bool thread_check` - flaga sterująca zakończeniem działania wszystkich wątków.
- `mutex print_mutex` - mutex do synchronizacji wypisywania komunikatów na konsolę.
- `Semaphore sem` - własna implementacja semafora licznikowego ograniczającego liczbę filozofów próbujących jeść (zalecane: `Semaphore sem(threads-1)`).

Kluczowe funkcje:

- `void philosopherLoop(const int id)` - funkcja wykonywana przez każdy wątek-filozofa. Zawiera główną pętlę: myślenie (losowy czas), wypisanie stanu głodu, próba zdobycia widelców (mutexy), jedzenie (czas jak myślenia), wypuszczenie widelców, powrót do myślenia. Zawiera także mechanizm wykrywania zagłodzenia (jeśli filozof czeka ponad 10 sekund na możliwość jedzenia, program zostaje przerwany).
- `int main()` - inicjalizuje zmienne globalne, uruchamia wątki filozofów, czeka określony czas trwania symulacji, a następnie kończy wszystkie wątki i dołącza je do głównego programu.

Synchronizacja, unikanie zakleszczenia i zagłodzenia:

- Każdy filozof przed próbą podniesienia widelców wywołuje `sem.try_acquire()` w pętli, co pozwala wykryć zagłodzenie (limit czasu oczekiwania).
- Widelce są blokowane przez wywołanie `std::lock(sticks[left], sticks[right])`, co zapobiega zakleszczeniu na poziomie mutexów.
- Po zakończeniu jedzenia mutexy są odblokowywane, a semafor zwalniany przez `sem.release()`.
- Wszystkie wypisy na konsolę są chronione przez `print_mutex`, aby uniknąć mieszania się komunikatów z różnych wątków.

Opis działania semafora:

- Klasa Semaphore wykorzystuje mutex i `condition_variable` do implementacji licznika dostępnych "pozwoleń".
- Metoda `acquire()` blokuje wątek, jeśli nie ma dostępnych pozwoleń, a `release()` je zwalnia i powiadamia czekające wątki.
- Metoda `try_acquire()` pozwala nieblokująco sprawdzić dostępność semafora (używana do wykrywania zagłodzenia).

Losowanie czasu:

- Czas myślenia każdego filozofa jest losowany z przedziału 1-5 sekund przy użyciu funkcji `rand()`.
- Czas jedzenia jest równy czasowi myślenia z danej iteracji.

2.5. Badania

W ramach badań przeprowadzono serię eksperymentów z różną liczbą filozofów (5, 10, 50, 100) oraz różnymi ustawieniami semafora ograniczającego liczbę filozofów próbujących jeść jednocześnie. Celem było zaobserwowanie, jak zmiana tych parametrów wpływa na występowanie zakleszczenia i zagłodzenia oraz na ogólne zachowanie programu. Czas każdej symulacji ustawiono na 60 sekund.

Liczba filozofów (n)	Ograniczenie semafora	Zagłodzenie	Zakleszczenie
10	$\lceil \frac{1}{3} \cdot n \rceil$	wystąpiło	brak
10	$\lceil \frac{1}{2} \cdot n \rceil$	wystąpiło	brak
10	$n - 1$	brak	brak
50	$\lceil \frac{1}{3} \cdot n \rceil$	wystąpiło	brak
50	$\lceil \frac{1}{2} \cdot n \rceil$	wystąpiło	brak
50	$n - 1$	brak	brak
100	$\lceil \frac{1}{3} \cdot n \rceil$	wystąpiło	brak
100	$\lceil \frac{1}{2} \cdot n \rceil$	wystąpiło	brak
100	$n - 1$	brak	brak

Tabela 2: Wyniki eksperymentów dla różnych parametrów symulacji

2.6. Wnioski

Przeprowadzone eksperymenty potwierdziły, że odpowiednie ustawienie ograniczenia semafora ma kluczowe znaczenie dla poprawnego działania algorytmu uczujących filozofów. Zbyt mała wartość semafora prowadzi do zagłodzenia niektórych wątków, natomiast ustawienie go na $n - 1$ (gdzie n to liczba filozofów) skutecznie eliminuje zagłodzenie. Mechanizm wykrywania zagłodzenia pozwala łatwo zidentyfikować niepoprawne ustawienia synchronizacji i potwierdzić poprawność działania programu. Zakleszczenie nie wystąpiło w żadnym z testów dzięki zastosowaniu semafora oraz odpowiedniej kolejności blokowania mutexów.

2.7. Podsumowanie

Problem uczujących filozofów jest klasycznym przykładem wyzwań związanych z synchronizacją w programowaniu współbieżnym. Przeprowadzona implementacja i badania pokazały, że stosując prosty semafor licznikowy oraz odpowiednie techniki synchronizacji, można skutecznie zapobiec zakleszczeniu i zagłodzeniu. Eksperymenty wykazały, że poprawne ustawienie parametrów synchronizacji jest kluczowe dla zapewnienia sprawiedliwego dostępu do zasobów przez wszystkie wątki. Opracowany program pozwala na łatwą obserwację i analizę działania algorytmów synchronizacyjnych w praktyce.

2.8. Literatura

- https://pl.wikipedia.org/wiki/Problem_uczuj%C5cych_filozof%C5%9Bw
- https://en.wikipedia.org/wiki/Dining_philosophers_problem
- Mirosław Zelent kurs C++ <https://miroslawzelent.pl/kurs-c++/>

2.9. Kod

```
1  #include <iostream>
2  #include <thread>
3  #include <vector>
4  #include <mutex>
5  #include <condition_variable>
6  #include <atomic>
7  #include <chrono>
8
9  using std::cout;
10 using std::vector;
11 using std::thread;
12 using std::mutex;
13 using std::lock_guard;
14 using std::this_thread::sleep_for;
15 using std::condition_variable;
16 using std::unique_lock;
17 using std::atomic_bool;
18 using std::chrono::seconds;
19
20 int simulation_time = 60;           // czas symulacji w sekundach
21 int threads = 11;                  // liczba filozofów i widelców
22 vector<thread> thread_vec;          // rezerwacja miejsca na wątki
23 vector<mutex> sticks(threads);      // widelce
24 atomic_bool thread_check = true;    // flaga kontrolująca działanie
   ↪   wątków
25 mutex print_mutex;                 // mutex do synchronizacji
   ↪   wypisywania na ekran
26 vector<int> meals;                  // licznik zjedzonych posiłków
   ↪   przez każdego filozofa
27 // stany filozofów
28 enum philoState {
29     Think,
30     Hungry,
31     Eat
32 };
33
34 // klasa ograniczająca liczbę filozofów próbujących jeść na raz
35 class Semaphore {
36     public:
37         bool try_acquire() {
38             unique_lock<mutex> lock(mtx, std::try_to_lock);
39             if (!lock.owns_lock() || count == 0) return false;
```

```

40         --count;
41         return true;
42     }
43     mutex mtx;
44     condition_variable cv;
45     int count;
46     explicit Semaphore(int initial) : count(initial) {}
47     void acquire() {
48         unique_lock<mutex> lock(mtx);
49         cv.wait(lock, [&] { return count > 0; });
50         --count;
51     }
52     void release() {
53         unique_lock<mutex> lock(mtx);
54         ++count;
55         cv.notify_one();
56     }
57 };
58 Semaphore sem(threads - 1);           // ograniczenie jedzenia na
    ↪ raz
59
60 void philosopherLoop(const int id) {
61     while (thread_check) {
62         // myślenie
63         int think_time = 1 + rand() % 5;
64         {
65             lock_guard<mutex> lock(print_mutex);
66             cout << "Filozof " << id << " myśli (" << think_time <<
    ↪ " s).\n";
67         }
68         sleep_for(seconds(think_time));
69         // głód
70         {
71             lock_guard<mutex> lock(print_mutex);
72             cout << "Filozof " << id << " jest głodny.\n";
73         }
74         time_t hungry_since = time(NULL);
75         while (true) {
76             if (!thread_check) return;
77             if (time(NULL) - hungry_since > 10) {
78                 lock_guard<mutex> lock(print_mutex);
79                 cout << "Filozof " << id << " został zagłodzony!\n";
80                 thread_check = false;
81                 return;
82             }
83             if (sem.try_acquire()) break;
84             sleep_for(seconds(1));
85         }
86         // próba podniesienia widelców

```

```

87         int left = id;
88         int right = (id + 1) % threads;
89         std::lock(sticks[left], sticks[right]);
90         // jedzenie
91         {
92             lock_guard<mutex> lock(print_mutex);
93             cout << "Filozof " << id << " je (" << think_time << "
                ↪ s).\n";
94         }
95         sleep_for(seconds(think_time));
96         // zakończony posiłek - odnotuj i odłóż widelce
97         ++meals[id];
98         sticks[left].unlock();
99         sticks[right].unlock();
100        sem.release();
101    }
102 }
103
104 int main() {
105     srand(static_cast<unsigned>(time(nullptr)));
106     thread_check.store(true);           // flaga kontrolująca
        ↪ działanie wątków
107     thread_vec.resize(threads);         // rezerwacja miejsca na
        ↪ wątki
108     meals.assign(threads, 0);
109     int i = 0;                          // id filozofa
110     for (thread& thr : thread_vec) {
111         thr = thread(philosopherLoop, i++);
112     }
113     sleep_for(seconds(simulation_time));
114     thread_check.store(false);           // sygnał zakończenia dla
        ↪ wątków
115     for (thread& thr : thread_vec) {
116         thr.join();
117     }
118     {
119         lock_guard<mutex> lock(print_mutex);
120         cout << "\nPodsumowanie posilkow:" << "\n";
121         for (int id = 0; id < threads; ++id) {
122             cout << "Filozof " << id << ": " << meals[id] << "
                ↪ posilkow" << "\n";
123         }
124     }
125     return 0;
126 }

```